

From Raw Data to Finished Paper: Building a Reproducible Research Pipeline

Doctoral Orientation Week

Dr. Victor van Pelt

WHU – Otto Beisheim School of Management

September 1, 2026

Opening Question

Let's say you wrote a research paper and your laptop died. How long would it take you to reproduce your analyses and results (figures, tables, and overviews) with a new laptop?

- 1 Less than one hour (easy!)
- 2 Between one hour and a day (It's possible but painful)
- 3 More than one day (I'd be stuck)

Why should YOU care about a reproducible research pipeline?

Storing data, code, and analyses can quickly turn into a mess:

 descriptives	17/01/2021 16:18	Rich Text Format
 descriptives	29/04/2020 17:40	LaTeX Source File
 descriptivetime	17/01/2021 16:19	Rich Text Format
 descriptivetime	13/01/2021 12:01	LaTeX Source File
 do file	25/01/2021 23:56	Stata Do-file
 do_clean	06/07/2023 15:47	Stata Do-file
 dyndoc	12/12/2022 17:39	Brave HTML Docu...
 effort	27/02/2020 16:28	Adobe Acrobat D...
 epq	17/01/2021 16:22	Rich Text Format
 epq	28/07/2020 12:12	LaTeX Source File
 epq	07/02/2023 13:30	Microsoft Excel 97...
 frequencies	17/01/2021 16:19	Rich Text Format
 frequencies	29/04/2020 17:40	LaTeX Source File
 gee	09/03/2020 11:33	LaTeX Source File
 gee	09/03/2020 11:33	Text Document

Story Time!



Other common situations:

- Code no longer works because the files it calls have moved.
- The number of observations in a regression does not make sense due to a bad initial merge of the data. When correcting the mistake, the results change dramatically.
- Someone suggests changing the sample definition. Updating it requires updating dozens of files because the code has been copied and pasted throughout the working directory.
- We want to collaborate with another researcher, but opening their directory reveals hundreds of code and files. Just figuring out which coding files to run in which order takes weeks. In the end, we give up and do everything from scratch.
- Working together in a directory on a cloud service causes everyone to be interrupted because changes one of us makes causes the other's code to break and produces conflicted copies.

But there is more at stake!

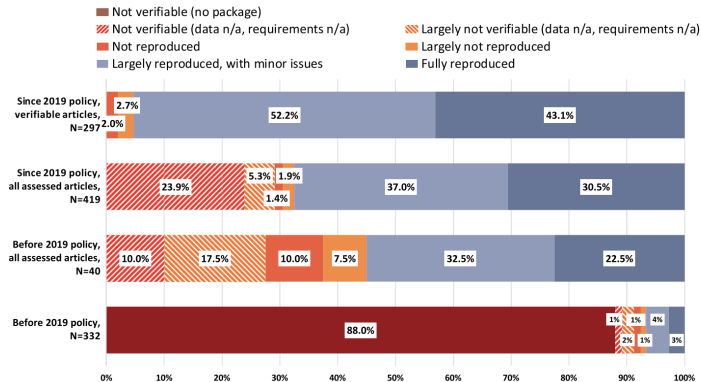
In 2019, *Management Science* introduced a Data and Code Disclosure Policy:

“Authors of accepted papers... must provide... the data, programs, and other details of the experiment and computations sufficient to permit replication.”

A 2024 study examined nearly 500 articles published before and after this policy was introduced to see if the results could, in fact, be reproduced (Fišar et al. 2024).

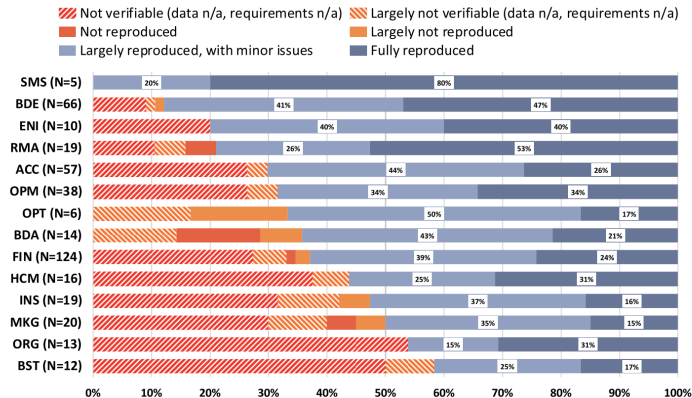
What share of these articles do you think were largely reproducible?

But there is more at stake!



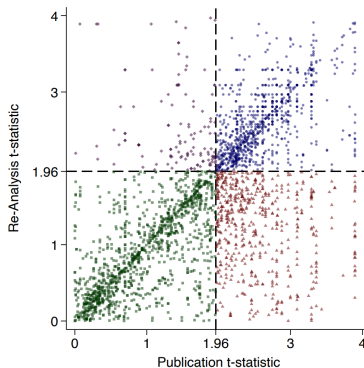
Before (after) the 2019 disclosure policy, 6.6% (67.5%) of articles were reproducible. Under the policy and for verifiable articles, 95.3% was reproducible!

But there is more at stake!



Considerable heterogeneity reproducibility across fields, mostly due to size, method, and data availability.

More recent evidence



Over 85% of articles in economics and political science journals with sharing policies were reproducible.
72% of significant estimates stay significant and same-signed (Brodeur et al. 2026).

Reproducible research starts with you

- Even when mandating verifiable research pipelines, roughly 15% to 33% of articles fails the reproducibility test.
- This is why a structured and sharable research pipeline is not only a formality or a “nice to have.”
- It's essential for credible research and for us to trust the insights it produces.
- It also does not only rest on professor's shoulders. It all starts with:



The main goals of this session:

- Sharing a few basic principles, techniques, and tools. Inspired by:
 - **Good Enough Practices in Scientific Computing** (Wilson et al.)
 - **The TIER Protocol** (Project Tier)
 - **Code and Data for the Social Sciences** (Gentzkow and Shapiro)
 - **How to keep your research projects organized** (de Kok)
- Please interrupt, ask questions, and share your experience.
- Disclaimer: Largely through the lens of a quantitative researcher.
- All materials for this talk are available: **Example Repository** and **These Slides**

What will we cover in this session?

- 1 Directory structure: How to organize your research project
- 2 Version control: Using Git and repositories (example using GitHub)
- 3 Integrating multiple engines: Example using Quarto
- 4 Some comments on AI in the research pipeline



1. Directory Structure

What are key principles of a reproducible research pipeline?

Universal principles:

- Anyone can run the code anywhere and anytime, regardless of location.
- Anyone can understand the code and data without too much effort.
- Anyone can change one number or one step and reproduce everything.

The key is to design and structure your process

- Use a flow-inspired directory structure:
 - E.g., formatting files -> generating variables -> conducting analyses -> generating output.
- Leave the raw data untouched: You load it and use it to produce output:
 - E.g., raw data files -> input files -> process files -> output files.
- Use relative paths (e.g., `../0_data/data.csv`) and not direct paths (e.g., `"C:/user[name]/Documents/research/project_1/0_data/data.csv"`).

Anyone should be able to delete everything except the raw data and the code, and rebuild your whole project.

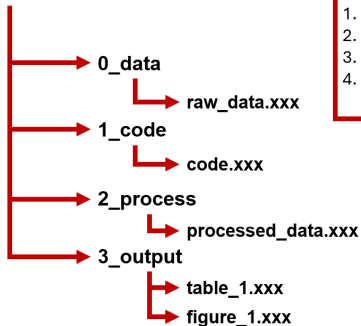
Use coding, so others can use and run it

- Make your code deterministic: set a seed for random and stochastic processes.
- Pin your environment: “*version*” in Stata, *renv* in R.
- Use plenty of comments to explain what the code is doing.
- Be as descriptive as you can in folder and file names; avoid special characters.
- Prefer tool-agnostic file formats: a “.csv” opens anywhere, for anyone.
- Use environment variables for credentials and software paths.

Write for the stranger who inherits the project, because in a few years that stranger is you.

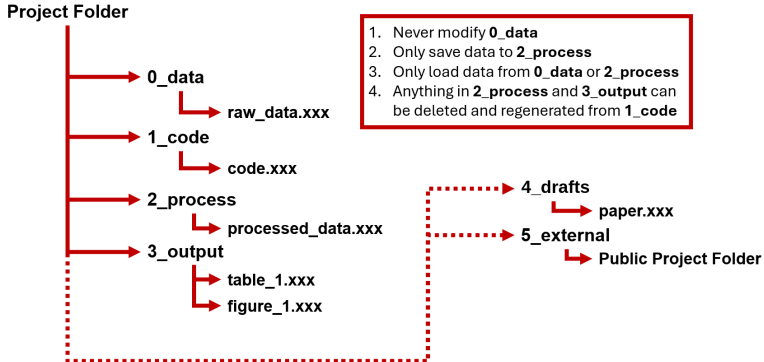
A simple starting point for a directory structure

Project Folder



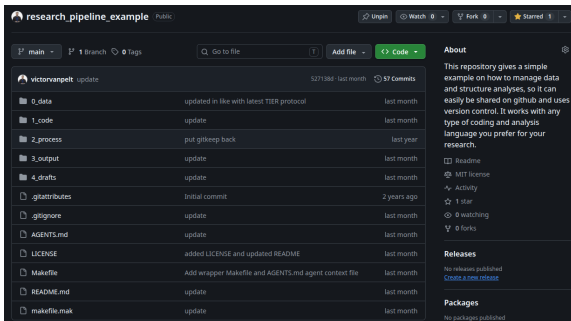
1. Never modify **0_data**
2. Only save data to **2_process**
3. Only load data from **0_data** or **2_process**
4. Anything in **2_process** and **3_output** can be deleted and regenerated from **1_code**

Additional folders and extensions



An example directory for you to use

Let's take a closer look at how this works in practice (using Stata, R, and Quarto):



Click here to use this repository!

Detailed overview of the example directory

research_pipeline_example/	
0_data/	raw inputs (real raw data is NOT shared)
gen_ai_earnings.csv	small synthetic dataset so the example runs
codebook.md	what each variable means and its type
1_code/	analysis code (shared; the heart of the repo)
code.do	Stata
code.r	R
code.qmd	Quarto version of the R code
2_process/	intermediate / passing data between steps (NOT shared)
3_output/	shared outputs: tables and figures
table_1.tex	written by the analysis, read by the drafts
data_appendix/	stats and figures for the data appendix
4_drafts/	the documents; each .qmd renders to a committed .pdf
paper.qmd	the manuscript
presentation.qmd	the slides
data_appendix.qmd	the data appendix (describes the analysis data)
references.bib	bibliography for the drafts
materials/	beamer theme, logo, fonts for the slides
makefile.mak	build tasks (the pipeline)
Makefile	two-line wrapper including makefile.mak
AGENTS.md	rules for AI coding agents working in this repo
.gitignore	what is and is not version-controlled
.gitattributes	normalizes line endings across operating systems

0_data: codebook.md

```
codebook.md
0_data > codebook.md > abc # Codebook: gen_ai_earnings.csv
1 # Codebook: gen_ai_earnings.csv
2
3 Synthetic example data, hand-authored for teaching. All values are
4 invented; no real firms or people. Reuse freely.
5
6 **Unit of observation:** one synthetic firm-observation per row (N =
7 200). Plain CSV with a header row.
8
9 | Variable | Type | Definition |
10 |-----|-----|-----|
11 | `id` | integer | Observation identifier (1-200) |
12 | `gen_ai` | indicator | 1 = uses generative AI, 0 = does not |
13 | `earnings` | numeric | Synthetic earnings measure, in arbitrary
14 units (negative values occur) |
15
16 If you replace the example with your own data, document it here the
17 same way: define every variable, and record the source, the citation,
18 the download date, and how someone else can obtain the file.
```

0_data/codebook.md lists what each variable in the raw data means and its type

1_code: Stata do-file (same analysis in code.r and code.qmd)

```
1_code/code.do (excerpt)

1 //clear everything to ensure nothing came before
2 clear all
3
4 //go to the project root
5 capture cd 1_code // if we're at the project root, step into 1_code (otherwise nothing)
6 cd .. // now we're guaranteed to be at the project root
7 // Forward slashes work on Windows, macOS, and Linux.
8
9 //import raw data and store an untouched copy in the process-folder (also handy for merging)
10 import delimited "0_data/gen_ai_earnings.csv", clear
11 save "2_process/gen_ai_earnings.dta", replace
12
13 //import the raw copy, transform the data, and save the edited data
14 use "2_process/gen_ai_earnings.dta", clear
15
16 gen earnings_scaled = earnings / 10
17 drop if earnings_scaled < 0
18
19 save "2_process/edit_gen_ai_earnings.dta", replace
20
21 //import the edited data and run the analysis to generate output
22 use "2_process/edit_gen_ai_earnings.dta", clear
23
24 //Run a basic regression with robust SE and store results (run "ssc install estout" once first!)
25 eststo: regress earnings_scaled gen_ai, r
26
27 //Write a LaTeX table to 3_output that the paper and slides can \input.
28 //booktabs produces \toprule/\midrule/\bottomrule, matching the R output.
29 //nonumbers drops esttab's default "(1)" header row and "coeflabels" renames
30 //"_cons" to "Constant", so this table matches the one code.r writes.
31 estab using "3_output/table_1.tex", replace booktabs ///
32 b(3) se(1) star(* 0.10 ** 0.05 *** 0.01) ///
33 stats(N r2 F p, fmt(0 3 3 3) labels("Observations" "\$R^2\$" "F" "p-value")) ///
34 nonumbers coeflabels(cons Constant) ///
35 title(TABLE 1 - REGRESSIONS OF SCALED EARNINGS ON GENERATIVE AI) ///
36 nonotes addnotes("Robust standard errors in parentheses. * p<0.10, ** p<0.05, *** p<0.01.")
37 eststo clear
```

3_output: Table 1 - Regression of Scales Earnings on Generative AI

	earnings_scaled
gen_ai	2.167*** (0.741)
Constant	5.781*** (0.554)
Observations	157
R^2	0.050
F	8.546
p-value	0.004

Robust standard errors in parentheses. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

How do you currently run analyses and produce results?

- 1 A code file run by hand in the right order
- 2 Point and click, from memory
- 3 Ask AI to produce results for me
- 4 Other

A structured pipeline allows you to use makefiles!

A makefile states which outputs depend on which code and data, and how to rebuild them

```
# =====  
# Research pipeline - build tasks  
#  
# Usage:  
# make      run the R analysis, then build the appendix, paper, slides  
# make r    run the R analysis      (1_code/code.r)  
# make stata run the Stata analysis  (1_code/code.do)  
# make quarto render the Quarto file (1_code/code.qmd)  
# make appendix render the data appendix (4_drafts/data_appendix.qmd)  
# make paper render the paper (4_drafts/paper.qmd)  
# make slides render the slides (4_drafts/presentation.qmd)  
# make clean delete everything the pipeline generates  
# make help list the targets  
#  
# Tools are overridable, e.g.: make r RSCRIPT=/path/to/Rscript  
#                               make stata STATA=StataSE-64 STATAFLAG=/e  
# Forward slashes work on every OS. On Windows, run this from Git Bash / WSL /  
# MSYS2, which provide make together with rm, mv, and find.  
# =====
```

makefile.mak in root

You run “*make*” in the CLI to rebuild everything, or a single step with “*make r*”, “*make stata*”, “*make quarto*”, “*make appendix*”, “*make paper*”, or “*make slides*”

Terminal – research_pipeline_example

```
1 $ make
2 Rscript --vanilla 1_code/code.r
3 quarto render 4_drafts/data_appendix_example.qmd
4 quarto render 4_drafts/paper_example.qmd
5 quarto render 4_drafts/presentation_example.qmd
```

Is your code slow?

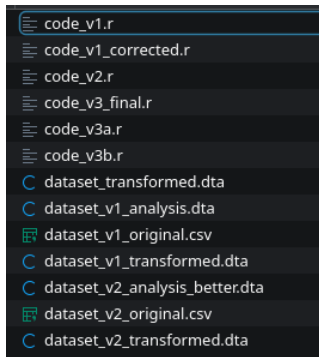
- Does your code:
 - Take a long time to run?
 - Require too much memory?
- Ask your supervisor for a better laptop.
- Otherwise, consider looking into running your code remotely:
 - A “supercomputer,” “grid,” or “server” is more powerful.
 - Ask about your school’s research computing services.



2. Version Control

Opening question:

Whose folders look something like this?



What is version control?

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.



How do I use version control?

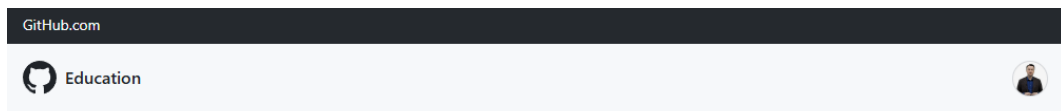
- Many cloud services have some forms of version control (e.g., Dropbox and OneDrive), but they can be unreliable and do not trace your work process properly.
- The most widely used software is called **git**
- You can run **git** locally using the command line (CLI), but it is easier to use an online provider to do everything for you.
- Saving your version control in the cloud reduces the likelihood you lose stuff, and allows you to document and track your work process properly.
- Three major **git** providers:
 - GitHub -> The best choice!
 - Bitbucket
 - GitLab

What is GitHub?

- GitHub hosts your working directory (a.k.a. your git repositories or “repos”) online and adds collaboration tools around them.
- Why use it?
 - Clean and easy-to-use interface
 - Works with any language and file type
 - GitHub Desktop application is very simple and convenient
- Bonus feature: you can use GitHub Pages to host your website for free! See mine **www.victorvanpelt.com**

GitHub Education

As a verified student, you get GitHub Pro and the Student Developer Pack for free **here**.



[Home](#) / [Benefits application](#)

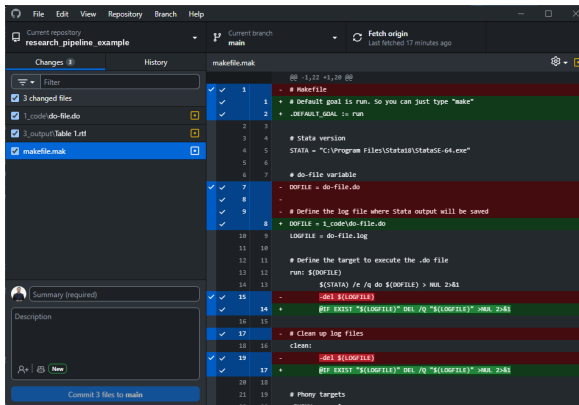
Unlock a passion for learning with GitHub

Complete the fields below to unlock GitHub Enterprise for your school

If you are looking for individual GitHub Education benefits for students or teachers, please apply [in your GitHub settings](#).

Download GitHub Desktop

Download GitHub Desktop for Windows or macOS [here](#).



What is the basic workflow with GitHub?

- First time:
 - Create a new repository using GitHub
 - Clone it to your computer or Publish it using GitHub Desktop
- When you start working:
 - Sync with GitHub first: “Pull” changes
- After making changes:
 - Sync with GitHub first: “Pull” changes
 - Create commit (add summary and descriptions)
 - Push commit to GitHub.
- Use a README.md file to state the project's title and a brief description of the repository.

.gitignore files

- There are lots of things you might not want to sync with GitHub
 - 1 Private or non-public data
 - 2 Personal credentials
 - 3 “Byproduct” files that aren’t critical to the repo
- Think carefully about what you sync with GitHub.
- A .gitignore file allows you to select files that should automatically be ignored by git.

.gitignore syntax

- The basic syntax can be found **here**.
- Some common expressions:
 - # -> comment
 - * -> any file
 - / at the end -> folder
 - ! -> keep (don't ignore)

.gitignore example

The example repository ignores everything by default, then allows back what should be shared:
the safest default for private data

```
.gitignore
1 # Ignore everything by default...
2 *
3
4 # ...but keep the project scaffolding
5 !.gitignore
6 !.gitattributes
7 !makefile.mak
8 !Makefile
9 !AGENTS.md
10 !README.md
11
12 # 0_data: keep the folder, the shared synthetic example CSV, and its codebook.
13 # Real raw data is intentionally NOT shared here (add it locally; it stays ignored).
14 !0_data/
15 !0_data/.gitkeep
```

Why should you care?

- Remember my anecdote at the start:
 - You need to be able to go back to any version at any time.
- Journals and institutions require access to your research materials (i.e., code, instrument, and data).
- At the very least, they want to ensure it exists.
- At the very most, they might check your code:
 - *Management Science* has verified reproducibility since 2019, partly through a third-party agency.
- Version control plus the directory structure earlier makes compliance nearly free.



3. Integrating Multiple Languages

What's a remaining challenge?



- Right now, you already have:
 - 1 A good directory structure
 - 2 Version control.
- A remaining challenge is that we use different software, coding languages, and files.
 - Python, R, Markdown, HTML, Office, Stata, LaTeX, etc.
 - Tables, datasets, code, and docs.
- There have been efforts over the past decade to put all processes under one umbrella.
- One system can integrate nearly everything into one process: Quarto.

What is Quarto?

- Quarto is an open-source scientific and technical publishing system
- Quarto is not only used for data. It can use data to produce a wide variety of outputs:
 - Articles and papers
 - Presentations (this presentation is made using Quarto)
 - Dashboards
 - Websites
- Runs R (Knitr), Python and Julia (Jupyter), and Observable JS next to Markdown and LaTeX.
- Quarto 2 (late 2026) stays backward compatible; your .qmd files keep working.

How to get started?

Setup:

- 1 Install Quarto from the website: <https://quarto.org/docs/get-started/>.
- 2 Choose an IDE or coding environment (I recommend **VS Code**).
- 3 Install the **Quarto VS Code Extension** in VS Code.

Usage:

- Quarto uses .qmd files. Check the **guide**.
- You can produce output in formats, such as .pptx, .docx, .pdf and integrate R and Stata code.
- The example repository uses Quarto to render a paper, a presentation, and a data appendix ("*4_drafts/*") that pull their numbers and tables straight from the pipeline.
- It also contains a self-contained literate report ("*1_code/code.qmd*").

Quarto Presentation Example

```
1_code/code.qmd
---
title: "Quarto Example: A Report Built at Render Time"
author: "Victor van Pelt"
format:
  beamer:
    theme: Madrid
    fontsize: 9pt
---
```

Specify in the "YAML" the type of
qmd file

```
1_code/code.qmd
---{r}
#| include: false

# Use CRAN mirror (needed when running via Rscript / make / quarto)
options(repos = c(CRAN = "https://cloud.r-project.org"))

# Move to the project root whether we render from the root or from 1_code.
# knitr resets the working directory after every chunk, so we store ROOT and
# build absolute paths with file.path(ROOT, ...) instead of relying on setwd().
if (dir.exists(file.path(getwd(), "1_code"))) setwd(file.path(getwd(), "1_code"))
setwd(".")
ROOT <- getwd()

need <- c("sandwich", "tibble", "modelsummary")
to_install <- setdiff(need, rownames(installed.packages()))
if (length(to_install)) install.packages(to_install) # repos taken from options()
invisible(lapply(need, require, character.only = TRUE))

# One self-contained pipeline: import -> transform -> analyse (no other file needed)
df <- read.csv(file.path(ROOT, "0_data", "gen_ai_earnings.csv"))
df$earnings_scaled <- df$earnings / 10
df <- df[df$earnings_scaled >= 0, , drop = FALSE]

mod <- lm(earnings_scaled ~ gen_ai, data = df)
vc <- sandwich::vcovHC(mod, type = "HC1") # robust SE, matches Stata's ", r"
---
```

Use R as you would in code.r

Quarto Presentation Example



Presentation Example

Victor van Pelt

WHU – Otto Beisheim School of Management
August 31, 2026

Import table

Table 1: TABLE 1 - REGRESSIONS OF SCALED EARNINGS ON GENERATIVE AI

	earnings_scaled
gen_ai	2.167*** (0.741)
Constant	5.781*** (0.554)
Observations	157
R^2	0.050
F	8.546
p-value	0.004

Notes. Robust standard errors in parentheses. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

2/4

One pipeline, three outputs (report, paper, slides), and the numbers always in sync.

makefile.mak extension

“*make appendix*”, “*make paper*”, and “*make slides*” render the Quarto drafts from the pipeline’s output; “*make quarto*” renders the self-contained report

```
makefile.mak (Quarto targets)
1  quarto:
2      ${QUARTO} render 1_code/code.qmd
3
4  # --- Manuscript, slides, and data appendix (consume 2_process and 3_output)
5  # Run 'make r' (or 'make stata') first so the inputs they read exist.
6  appendix:
7      ${QUARTO} render 4_drafts/data_appendix_example.qmd
8
9  paper:
10     ${QUARTO} render 4_drafts/paper_example.qmd
11
12  slides:
13     ${QUARTO} render 4_drafts/presentation_example.qmd
14
15  # --- Housekeeping -----
16  # Delete everything in 2_process and 3_output except the .gitkeep files,
17  # plus stray render artifacts, so a fresh 'make' reproduces every output.
18  clean:
19      @find 2_process 3_output -type f ! -name '.gitkeep' -delete
20      @rm -f 1_code/code.pdf 4_drafts/*.pdf *.log 1_code/*.log
21      @echo "Cleaned 2_process, 3_output, and render artifacts."
22
23  help:
24      @echo "Targets: all (default), r, stata, quarto, appendix, paper, slides, clean"
```



4. AI in your research pipeline

How will AI affect the research pipeline and reproducibility?

Somewhere around 15-33% of articles in journals with mandatory sharing policies cannot be reproduced.

Question: How will AI affect reproducibility?

AI can hurt, if you over-delegate

Consider this spectrum:



Where does AI assistance end and replacement begin for you?

Examples of over-delegating research to AI

- A dataset, table, or coefficient directly generated by AI
- Submitting confidential or licensed data to AI providers
 - Sometimes you can't even use local llms!
- Direct edits to raw data, processing files, or output files
- Invented numbers and code that make no sense

When AI takes over, research steps can be lost

- Raw data should stay untouched in “0_data”, while “2_processing” and “3_output” are only generated by files in “1_code”.
- Once AI cleans, codes, merges, or edits a data file, we cannot directly trace what it did
- So, we don't want AI to directly touch anything, except what's in “1_code”.
- What if AI generates data, like creating or structuring a variable?
 - If AI generates data, it should be treated as input, like the data in “0_data”.

Ok, but what CAN AI do for us?

- Everything we need:
 - data cleaning, structuring, and merging
 - analyses and the production of tables and figures
 - reporting statistics in papers and slide decks.
- You must always check what changed and stay in control
- You also stay in the loop through relevant gates.

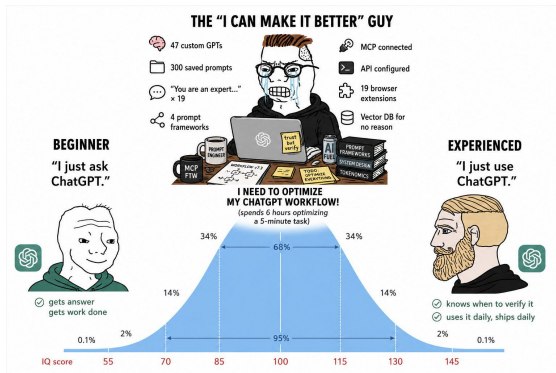
Terminal – an agent working in research_pipeline_example

```
1 $ claude "Add a robustness check: winsorize scaled
2 earnings at the 1st/99th percentiles before
3 the main regression."
4
5 * Reading 1_code/code.r
6 * Editing 1_code/code.r (+5 lines)
7 * Running: make clean && make
8   Rscript --vanilla 1_code/code.r
9   quarto render 4_drafts/data_appendix_example.qmd
10  quarto render 4_drafts/paper_example.qmd
11  quarto render 4_drafts/presentation_example.qmd
12
13 Done. Review my change with: git diff 1_code/code.r
```

As long as we can trace the research process, we can be responsible

How can we ensure agents follow such rules?

In the engineering space, it is common to integrate AI tooling directly in repo's. But AI changes rapidly and everyone wants to bring their tooling or not at all:



My suggestion: A shared rulebook (e.g., AGENTS.md)

AGENTS.md — context for AI coding agents

This repository is a template for a reproducible research pipeline. Data flows through numbered folders; everything downstream of the raw data is regenerated by code.

Rules

- Never modify the raw data in `@_data/`. It is read-only. The exception is `@_data/codebook.md`, which is hand-maintained documentation: update it whenever the data change.
- Never edit files in `2_process/` or `3_output/` by hand (including `3_output/data_appendix/`): they are build artifacts, regenerated by the pipeline.
- Help the user make changes in `1_code/` (analysis), `4_drafts/` (paper, slides, and data appendix sources), and `@_data/codebook.md` only.
- Before a change that touches more than one file, say which files you will edit and what will change in each. Then edit.
- Never run `git commit` or `git push`. Leave changes in the working tree for the maintainer to review and commit.
- Do not commit credentials; use environment variables.
- Remove any scratch or helper files you create in this repository after you're finished. The folder structure is fixed; do not add folders or top-level files.

How does this work?

Example:

- Prompt: winsorize scaled earnings at 1st/99th percentile
- The agent edits “1_code/code.r” and reproduces everything with *make*
- You check the difference in the GitHub app before accepting the change
 - Check: only “1_code” touched, and the step sits before the regression
 - Check: “0_data” and “3_output” untouched?
 - Check: did the results change the way you expected?
- You commit and push the change

Summary and Wrap-up

- Reproducibility has been under threat (Open Science Collaboration 2015; Baker 2016; Camerer et al. 2016; Hail, Lang, and Leuz 2020; Fišar et al. 2024; Brodeur et al. 2026)
- A research pipeline is vital to ensure results are reproducible.
- You, as WHU doctoral students, are vital:
 - 1 Maintain a good directory structure
 - 2 Use version control
 - 3 Integrate multiple languages (try Quarto)
 - 4 Let AI help you code not take over the research
- If you don't want to do it for science, then do it to save yourself lots of time and headache.

Download Example Research Pipeline

Download Slides

A research pipeline is just the start...

- Ideally, we also need:
 - Research material sharing (data, instruments, and code)
 - Pre-registration
- My recommendation is to try:
 - **AsPredicted**
 - **AsCollected**
 - **ResearchBox**
- To facilitate replication: **README Template**

Thank you!

Dr. Victor van Pelt
Professor of Behavioral Accounting
Finance and Accounting Group
WHU – Otto Beisheim School of Management

Campus Vallendar, Burgplatz 2, 56179 Vallendar, Germany
Tel.: +49 (0)261 6509 483
Victor.vanPelt@whu.edu
<https://www.victorvanpelt.com>



References I

- Baker, Monya. 2016. "1,500 Scientists Lift the Lid on Reproducibility." *Nature* 533 (7604): 452–54.
<https://doi.org/10.1038/533452a>.
- Brodeur, Abel, Derek Mikola, Nikolai Cook, Lenka Fiala, Thomas Brailey, Ryan Briggs, Alexandra de Gendre, et al. 2026. "Reproducibility and Robustness of Economics and Political Science Research." *Nature* 652 (8108): 151–56.
<https://doi.org/10.1038/s41586-026-10251-x>.
- Camerer, Colin F, Anna Dreber, Eskil Forsell, Teck-Hua Ho, Jürgen Huber, Magnus Johannesson, Michael Kirchler, et al. 2016. "Evaluating Replicability of Laboratory Experiments in Economics." *Science* 351 (6280): 1433–36.
<https://doi.org/10.1126/science.aaf0918>.
- Fišar, Miloš, Ben Greiner, Christoph Huber, Elena Katok, and Ali I. Ozkes. 2024. "Reproducibility in Management Science." *Management Science* 70 (3): 1343–56. <https://doi.org/10.1287/mnsc.2023.03556>.
- Hail, Luzi, Mark Lang, and Christian Leuz. 2020. "Reproducibility in Accounting Research: Views of the Research Community." *Journal of Accounting Research* 58 (2): 519–43. <https://doi.org/10.1111/1475-679X.12305>.
- Open Science Collaboration. 2015. "Estimating the Reproducibility of Psychological Science." *Science* 349 (6251): aac4716.
<https://doi.org/10.1126/science.aac4716>.